ISIMM

Programmation Python 1

CPI1

2026

Plan

01 Import de bibliothèques ou modules

02 Les sous-programmes

03 Gestion des exceptions

04 Gestion des fichiers

01 Import de bibliothèques ou modules

Il y a trois manières d'import de modules :

- **1ère manière d'import**

  Import du module désigné par nom_module mais pas les fonctions qui y sont contenues :

```
import nom_module
```

Exemples :

```
import math # import du module math
import random # import du module random
```

01 Import de bibliothèques ou modules

L’appel de chaque fonction du module se fait par :

nom_module.fonction(...)

Exemples :

math.ceil(7.8989) # partie entière supérieure qui est 8

random.randint(0,10) # retourne un entier aléatoire entre 0 et 10

01 Import de bibliothèques ou modules

- **2ème manière d'import**

  Utilisation d'un alias : import nom_module as alias

Exemple1 :

```
import math as m
```

L'appel de chaque fonction du module se fait par :

```
alias.fonction(...)
```

Exemple 2:

```
m.sqrt(4) #retourne la racine carrée de 4 : 2.0
```

01 Import de bibliothèques ou modules

- **3ème manière d'import**

- Accès direct à toutes les fonctions du module par :

from nom_module import *

Exemple: from math import *

- Accès direct à une ou quelques fonctions du module par :

from nom_module import nom_fonc1,...,nom_foncp

Exemple: from math import pi,sqrt,ceil

Dans les deux cas, l'appel de chaque fonction du module se fait directement par son nom : nom_fonction(...)

Exemple: sqrt(5) #donne 2.23606797749979

02 Les sous-programmes

- Une fonction est un bloc d’instructions indentées (corps de la fonction) destiné à réaliser une opération. Elle peut recevoir des paramètres et renvoie éventuellement un résultat. L’appel d’une fonction Python retourne toujours un objet, par défaut None. Dans le corps de la fonction ce retour est spécifié à l’aide de l’instruction return.

02 Les sous-programmes

Pourquoi utiliser une fonction ?

- Éviter la répétition du code
- Organiser le programme
- Rendre le code plus lisible
- Faciliter la réutilisation

```
def nom_fonction(parametres):
    instructions
    return valeur
```

02 Les sous-programmes

Description	Syntaxe	Exemple(s) / Remarques

Définition d'une fonction où les paramètres sont optionnels	def nom_f(paramètres) :
	corps de nom_f
		#fonction avec paramètres
		def f(x,y) :
		import math
		return x**2 + math.sin(y)
		#fonction sans paramètres
		def g() :
		while True:
		s = input("donner une chaîne : ")
		if s.isalpha():
		return s

Appel d'une fonction :
- Arguments positionnels
- Arguments nommés (ordre des arguments n'a plus d'importance)
Les paramètres nommés doivent apparaître à la fin.
	nom_f(arguments)
	
	nom_f(n_arg1=v1, …, n_argp=vp)
		#arguments positionnels
		print(f(3,2))
		
		#arguments nommés
		r = f(y = 2, x = 3)

Retour d'une fonction: L'instruction return permet de retourner un résultat à l'appelant d'une fonction.	return expressionUne fonction sans return retourne l'objet None.	#Définition def f(x) : if x > 0 : return x**0.5 #Appel print(f(4)) # affiche 2.0 print(f(-4)) # affiche None

02 Les sous-programmes

Description	Syntaxe	Exemple(s) / Remarques
Fonctions lambda: Elle permet de créer une expression de type fonction.	lambda paramètres : résultat	f = lambda x : x**2 #équivalente à def f(x) : return x**2

02 Les sous-programmes

- Paramètres d'Entrée (PE)
  - Variables passées à la fonction
  - Leur valeur ne change pas dans le programme principal
- Paramètres de Sortie (PS)
  - Variables créées dans la fonction
  - Toujours retournées avec *return*
- Paramètres Entrée/Sortie (PE/S)
  - Variables passées à la fonction
  - Modifiées dans la fonction

```
def somme (L):
    s = 0
    for e in L:
        s += e
    return s
```

02 Les sous-programmes

- Fonction sans return

```
def modifier(L):
    for i in range(len(L)):
        L[i] += 1
```

- Programme principal :

```
L = [5, 0, 8]
modifier(L)
print(L)
```

- Fonction return

```
def somme(L):
    return sum(L)
```

- Programme principal :

```
s = somme (L)
print(s)
```

- Paramètres positionnels **VS.** paramètres par mots-clés

Il existe deux types de paramètres que l'on peut donner à une fonction en Python :

– les paramètres positionnels ;
– les paramètres par mots-clés.

Contrairement aux paramètres positionnels, les paramètres par mot clé ont une valeur attribuée par défaut. On parle de paramètre formel pour désigner les paramètres de la fonction (les variables utilisées dans le corps de la fonction) et de paramètre effectif pour désigner la valeur que l'on souhaite donner au paramètre formel.

Pour définir la valeur à donner à un paramètre formel, on utilise le symbol d'égalité. Lors de l'appel de la fonction, si l'utilisateur ne définit pas explicitement une valeur, celle par défaut sera affectée. Ainsi, il n'est pas forcément nécessaire de préciser les paramètres par mots-clés lors de l'appel de la fonction.

Il est important de noter que les arguments positionnels (ceux qui n'ont pas de valeur par défaut) doivent apparaître en premier dans la liste des paramètres.

Les sous-programmes

Exemple:

```
def production_2(l, m, k=42):
    """  Retourne la valeur de la production en fonction  du travail, du capital et du matériel.
    Keyword arguments:
    l -- travail (float)  m -- matériel (float)
    k -- capital (float) (default 42)  """

    return l**0.3 * k**0.5 * m**(0.2)
```

La fonction production_2() peut s'appeler, pour donner le même résultat, des manières suivantes :

```
# En nommant tous les paramètres, en ommettant k
prod_val_1 = production_2(l = 42, m = 40)
# En nommant tous les paramètres et en précisant k
prod_val_2 = production_2(l = 42, m = 40, k = 42)
# En nommant uniquement le paramètre mot-clé k
prod_val_3 = production_2(42, 40, k = 42)
# En ne nommant aucun paramètre
prod_val_4 = production_2(42, 40, 42)
res = [prod_val_1, prod_val_2, prod_val_3, prod_val_4]print(res)
```

Exemple:

```
## [41.59215573604822, 41.59215573604822, 41.59215573604822, 41.59215573604822]
```

Remarque:

Si la fonction contient plusieurs paramètres positionnels ; lors de l'appel :

- soit on nomme tous les paramètres positonnels par leur nom ;
- soit aucun ;
- il n'y a pas d'entre deux.

- Portée

– Lorsque une fonction est appelée, le corps de cette fonction est interprété. Les variables ayant été définies dans le corps de la fonction sont assignées à un namespace local. C'est-à-dire qu'elles ne vivent qu'à l'intérieur ce cet espace local, qui est créé au moment de l'appel de la fonction et détruit à la fin de celui ci.
→ On parle alors de portée des variables.

– Une variable ayant une portée locale (assignée dans l'espace local) peut avoir le même nom qu'une variable globale (définie dans l'espace de travail global), sans pour autant désigner le même objet, ou écraser cet objet.

Exemple:

```
# Définition d'une variable globale :
valeur = 1# Définition d'une variable locale à la fonction f
def f(x):
        valeur = 2  nouvelle_valeur = 3
        print("valeur vaut :", valeur)
        print("nouvelle_valeur vaut :", nouvelle_valeur)
        return x + valeur
```

- Portée

– Appelons la fonction f(), puis regardons la valeur de valeur et celle de nouvelle_valeur après l'exécution de la fonction.

```
res = f(3)
## valeur vaut : 2
## nouvelle_valeur vaut : 3
print("valeur vaut :", valeur)
## valeur vaut : 1
print("nouvelle_valeur vaut :", nouvelle_valeur)
## NameError: name 'nouvelle_valeur' is not defined
```

→ Durant l'appel, la variable locale du nom valeur valait 2. Cette variable ne faisait pas référence à la variable du même nom définie dans l'environnement global. À l'issue de l'exécution de la fonction f(), cette variable valeur locale est supprimée, et il en est de même pour la variable locale nouvelle_valeur, qui n'existe pas dans l'environnement gloabl (d'où l'erreur retournée).

Les sous-programmes

- Portée

Principes de la portée des variables:
– Les variables sont définies dans des environnements, qui sont embriqués les uns dans les autres. Si une variable n'est pas définie dans le corps d'une fonction, Python ira chercher dans un environnement parent.

```
valeur = 1
def f(x):
    return x + valeur  print(f(2))
```

– Si on définit une fonction à l'intérieur d'une autre fonction, et qu'on appelle une variable non définie dans le corps de cette fonction, Python ira chercher dans l'environnement directement supérieur. S'il ne trouve pas, il ira chercher dans l'environnement encore supérieur, et ainsi de suite jusqu'à l'environnement global.
La variable valeur n'est pas définie dans # l'environnement local de g().

```
valeur = 1
def f():
    valeur = 2
    def g(x):
        return x + valeur
    return g(2)
print(f())
```

Les sous-programmes

- Portée

– La variable valeur n'est définie ni dans g() ni dans f()# mais dans l'environnement supérieur (ici, global)

```
valeur = 1
def f():
    def g(x):
        return x + valeur return g(2)
print(f())
```

– Si on définit une variable dans le corps d'une fonction et que l'on souhaite qu'elle soit accessible dans l'environnement global, on peut utiliser le mot-clé global :

```
def f(x):
  global y
  y = x+1
f(3)
print(y)
```

Remarque:

La variable que l'on souhaite définir de manière globale depuis un espace local de la fonction ne doit pas avoir le même nom d'un des paramètres.

03 Gestion des exceptions

- Les exceptions sont des objets spéciaux permettant de représenter une erreur d’exécution dans un script Python.
- Il existe plusieurs types d’erreurs : TypeError, ValueError, IndexError. Ces derniers sont des classes Python héritant toutes de la classe de base Exception permettant de représenter une Erreur quelconque.

03 Gestion des exceptions

Description	Syntaxe	Exemple(s) / Remarque(s)
Déclencher une exception	raise Exception(" message ")	raise Exception()
Prévoir une exception	if cond_erreur : raise Exception("message") #ou bien assert not cond_erreur, "message"	

Intercepter une exception	try :
	bloc
	except :
	bloc_cas_erreur
	else :
	bloc_cas_pas_erreur
	finally :
	bloc_tous_les_cas
		#saisie d'un entier positif (avec gestion d'erreur)
		
		def f():
		while True:
		try: x=int(input('x'))
		except: print ('erreur')
		else:
		if x>0:
		return x

04 Gestion des fichiers

Un fichier est une collection d'informations, généralement structurées d'une manière ou d'une autre, placées sur une mémoire de masse.

1. Fichiers textes

Un fichier texte contient une succession de caractères séquentiels. Lors de l'ouverture d'un fichier texte, un encodeur assure la conversion des octets lus/écris en lettres UNICODE.

04 Gestion des fichiers

Fonctions /Méthodes	Description/Exemples
f=open(nom_du_fichier,'r')	Ouvre le fichier nom_de_fichier (donné sous la forme d'une chaîne de caractères indiquant son emplacement) en lecture (r comme read). Le fichier doit exister et seule la lecture est autorisée. L'ouverture d'un fichier inexistant génère une erreur. L'utilisation de la clause try … except est indiquée pour gérer cette situatio
f=open(nom_du_fichier,'w')	Ouvre le fichier nom_de_fichier en écriture (w comme write). Si le fichier n'existe pas, il est créé, sinon il est écrasé. f est le fichier logique associé au fichier physique donné par nom_du_fichier.
f=open(nom_du_fichier,'a')	Ouvre le fichier nom_de_fichier en ajout (a comme append). Identique au mode 'w', sauf que si le fichier existe, il n'est pas écrasé et l'ajout se fait en fin du fichier.
f.close()	Ferme un fichier ouvert par la fonction open.
f.read()	Fournit l'intégralité du contenu du fichier sous forme de chaîne de caractères (à ne réserver qu'aux fichiers de taille raisonnable).

04 Gestion des fichiers

Fonctions /Méthodes	Description/Exemples
f.readlines()	Fournit la totalité du contenu du fichier sous forme d'une liste de chaînes de caractères. Le caractère de saut de ligne '\n' présent à la fin de chaque chaîne indique la fin de chaque ligne.
f.readline()	Fournit la ligne courante sous forme d'une chaîne de caractères et place le curseur sur la ligne suivante. Elle renvoie la chaîne vide si fin du fichier.
f.write(s)	Écrit la chaîne s à partir de la position courante du fichier.
f.writelines(T) T doit être une chaîne de sous chaînes ou un itérable de chaînes	Écrit l'ensemble des éléments de T dans le fichier f comme des lignes successives. Le caractère de saut de ligne '\n' présent à la fin de chaque chaîne indique la fin de chaque ligne.

04 Gestion des fichiers

On n'est jamais à l'abri d'une erreur quand on manipule des fichiers et qu'on oublie de les fermer. Le mot réservé ***with*** du langage python, permet d'éviter cette situation et dont voici sa syntaxe générale :

```
with open (mon_fichier, mode) as variable : # Opérations de manipulation du fichier
```

L'exemple précédent s'écrit alors :

```
try:
    with open ('fp.txt','r') as f :
        for ligne in f:
            print(ligne)
except:
    print(" fichier introuvable !")
```

2. Fichiers binaires

- Dans un fichier binaire les informations sont transférées de la RAM vers le support de stockage sans aucun formatage (aucun encodeur n’est utilisé). Pour les fichiers binaires, le mode d’ouverture est suffixé par 'b' : 'wb', 'rb' et 'ab'.
- La manipulation d'un fichier binaire n'est pas une fin en soi et cela ne présente aucun intérêt pour l'utilisateur. L'intérêt à ce niveau est de pouvoir utiliser le module **pickle** permettant de sérialiser/désérialiser des objets (entier, réel, chaîne, liste, dictionnaire, etc.) dans leur format et non pas sous forme de suite de caractères.
- Sérialiser un objet, à l'aide de la méthode dump, consiste à le transformer en une suite d’octets afin de pouvoir le stocker dans un fichier binaire. Désérialiser, à l'aide de la méthode load, consiste à récupérer les objets dans leur format initial.

2. Fichiers binaires

```
>>>import pickle
>>>pickle.dump(objet, fichier_binaire)
>>>objet = pickle.load(fichier_binaire)
```

- Remarque : load génère une erreur si plus d'objets à lire, l'utilisation du try ... except s'impose.

Exercice 1 :

On dispose d’un fichier texte nommé produits.txt contenant :

Ordinateur,1200
Souris,25
Clavier,abc
Ecran,300
Imprimante,-150

Chaque ligne contient :

NomProduit, Prix

Certaines valeurs peuvent être invalides.

Exercice 1 :

1) Écrire une fonction lire_produits(nom_fichier) qui :

- Prend en paramètre le nom d'un fichier.
- Lit le fichier ligne par ligne.
- Ignore les lignes dont le prix n'est pas numérique ou est négatif.
- Retourne une liste de tuples contenant uniquement les produits valides sous la forme :

```
[("Ordinateur", 1200.0), ("Souris", 25.0), ("Ecran", 300.0)]
```

- Gère les exceptions suivantes :
  - FileNotFoundError
  - ValueError

Exercice 1 :

2) Écrire une fonction calculer_statistiques(produits) qui :

- Prend en paramètre une liste de tuples (nom, prix).
- Utilise un itérable pour extraire les prix.
- Retourne :
  - le prix moyen
  - le prix maximum
  - le prix minimum

3) Écrire une fonction ecrire_resultats(nom_fichier, moyenne, maximum, minimum) qui :

- Écrit les résultats dans un fichier texte.
- Gère les exceptions possibles lors de l’écriture.

4) Écrire le programme principal qui :

- Importe le module statistics
- Appelle les fonctions précédentes
- Affiche un message si aucun produit valide n’est trouvé

Exercice 1 - Corrigé:

```
import statistics
def lire_produits(nom_fichier):
    produits = []
    try:
        with open(nom_fichier, "r") as fichier:
            for ligne in fichier:   # itérable
                try:
                    nom, prix = ligne.strip().split(",")
                    prix = float(prix)
                    if prix >= 0:
                        produits.append((nom, prix))
                except ValueError:
                    print("Ligne invalide ignorée :", ligne.strip())
        return produits
    except FileNotFoundError:
        print("Fichier introuvable.")
        return []
```

Exercice 1 - Corrigé:

```
def calculer_statistiques(produits):
    prix = [p[1] for p in produits]   # itérable (liste)
    moyenne = statistics.mean(prix)
    maximum = max(prix)
    minimum = min(prix)
    return moyenne, maximum, minimum

def ecrire_resultats(nom_fichier, moyenne, maximum, minimum):
    try:
        with open(nom_fichier, "w") as fichier:
            fichier.write(f"Moyenne : {moyenne}\n")
            fichier.write(f"Maximum : {maximum}\n")
            fichier.write(f"Minimum : {minimum}\n")
    except Exception as e:
        print("Erreur d'écriture :", e)
```

Exercice 1 - Corrigé:

```
# Programme principal
produits = lire_produits("produits.txt")

if produits:
    moyenne, maximum, minimum = calculer_statistiques(produits)
    ecrire_resultats("resultats.txt", moyenne, maximum, minimum)
    print("Analyse terminée avec succès.")
else:
    print("Aucun produit valide trouvé.")
```